

LINQ Prepend Method in C# With an Example

In this article, I will discuss the **LINQ Prepend Method in C#** with Examples. Please read our previous article before proceeding to this article, where we discussed the [LINQ Append Method](#) with an example.

Linq Prepend Method in C#:

The Prepend method in LINQ adds a single element to the beginning of a sequence. Like the Append method, Prepend is a part of LINQ and is used with `IEnumerable<T>` sequences. It was introduced in .NET Framework 4.7.1 and .NET Core 1.0. Like the Append method, the Prepend method does not modify the sequence elements. Instead, it creates a copy of the sequence with the new element. The signature of this is given below.

```
public static IEnumerable<TSource> Prepend<TSource>(this IEnumerable<TSource> source,
TSource element);
```

Type Parameters

1. **TSource**: The data type of the elements contained in the sequence.

Parameters:

1. **IEnumerable<TSource> source**: A sequence of values.
2. **TSource element**: The value to prepend at the beginning of the sequence.

Returns:

1. **IEnumerable<TSource>**: A new sequence that begins with the element.

Exceptions: When the source is null, it will throw `ArgumentNullException`.

Note: This method is supported by Framework 4.7.1 or later.

Example:

The following example shows how to prepend a value to the beginning of the sequence using the Prepend method. The following example code is self-explained. So, please go through the comment lines.

```
using System.Linq;
using System.Collections.Generic;
using System;
namespace LinqDemo
{
    class Program
    {
```

```

static void Main(string[] args)
{
    // Creating a list of numbers
    List<int> numberSequence = new List<int> { 10, 20, 30, 40 };
    // Trying to prepend 50
    numberSequence.Prepend(50);
    // It will not work because the original sequence has not been changed
    Console.WriteLine(string.Join(", ", numberSequence));
    // It works now because we are using a changed copy of the original sequence
    Console.WriteLine(string.Join(", ", numberSequence.Prepend(50)));
    // If you prefer, you can create a new list explicitly
    List<int> newnumberSequence = numberSequence.Prepend(50).ToList();
    // And then write to the console output
    Console.WriteLine(string.Join(", ", newnumberSequence));
    Console.ReadKey();
}
}
}

```

Output:

```

10, 20, 30, 40
50, 10, 20, 30, 40
50, 10, 20, 30, 40

```

Prepend is particularly useful when you're working with immutable sequences or when you're chaining multiple LINQ methods together, and you need to insert an element at the start of the sequence.

The Prepend method is similar to Append in that it does not modify the original sequence but instead returns a new `IEnumerable<T>` that, when iterated over, will start with the prepended element followed by the elements of the original sequence.

As with most LINQ methods, Prepend uses deferred execution, which means that the prepending does not occur until you enumerate over the sequence. If you want to realize the results into a list or array immediately, you can call `ToList` or `ToArray` on the enumerable:

```

using System.Linq;
using System;
namespace LinqDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            int[] numbers = { 1, 2, 3, 4, 5 };
            // Prepend 0 to the sequence of numbers
            var result = numbers.Prepend(0).ToArray();
            foreach (var number in result)
            {

```

```

        Console.WriteLine(number);
    }
    Console.ReadKey();
}
}
}

```

Remember that repeatedly using Prepend in a loop could lead to performance issues because it will create a new sequence in each iteration. If you need to prepend multiple elements, using a different approach may be more efficient, such as creating a list and using the Insert method.

When to use the LINQ Prepend Method in C#?

You should use the LINQ Prepend method in C# when adding an element to the beginning of a sequence or collection without modifying the original collection. Here are some scenarios where the Prepend method is particularly useful:

Building Sequences Step by Step: When constructing a sequence or collection element by element, you can use Prepend to add elements one at a time at the beginning of the sequence.

```

var sequence = Enumerable.Empty<int>();
sequence = sequence.Prepend(3);
sequence = sequence.Prepend(2);
sequence = sequence.Prepend(1);
// Result: [1, 2, 3]

```

Inserting a Header or Default Value: You can use Prepend to insert a header or default value at the beginning of a sequence, which is useful when preparing data for display or processing.

```

var data = GetDataFromDatabase();
data = data.Prepend(new HeaderItem());

```

Creating a New Sequence: When creating a new sequence that starts with a specific element, you can use Prepend without modifying the original sequence.

```

var originalSequence = Enumerable.Range(2, 4); // [2, 3, 4, 5]
var newSequence = originalSequence.Prepend(1); // [1, 2, 3, 4, 5]

```

Reversing a Sequence: To reverse the order of elements in a sequence, you can use Prepend to add elements from another sequence or collection in reverse order to a new sequence.

```

var originalSequence = new List<int> { 4, 3, 2, 1 };
var reversedSequence = originalSequence.Aggregate(Enumerable.Empty<int>(), (acc, x) => acc.Prepend(x));

```

Conditional Prepending: When you need to conditionally add an element to the beginning of a sequence based on certain conditions, the Prepend method allows you to do so readably.

```

var sequence = Enumerable.Empty<int>();
if (condition)

```

```
{  
    sequence = sequence.Prend(1);  
}
```

Important Considerations When Working With LINQ Prepend Method:

- Like most LINQ methods, Prepend uses deferred execution, so the element isn't actually prepended until the sequence is enumerated.
- Using Prepend in a loop for multiple elements can be inefficient as it creates a new sequence on each iteration. If you need to prepend multiple elements, consider using a different approach, like adding the elements to a list and reversing it, or using Concat if you already have a sequence of the elements to prepend.
- Prepend is only available in .NET Framework 4.7.1 and later, .NET Core, and .NET Standard 2.0 and later. If you're working with an earlier version, you must use alternative methods to achieve similar results.

So, the Prepend method is useful when you want to add an element to the beginning of a sequence or construct a new sequence while keeping the original sequence intact. It is particularly valuable in scenarios where you need to build sequences step by step, insert default values or headers, or reverse the order of elements in a sequence.

In the next article, I will discuss the [LINQ ZIP Method](#) with an example. In this article, I try to explain the Linq **Prepend Method in C#** with an example.